Docker & Kubernetes for Developers

Session 04: Kubernetes Hizmet Keşfi ve Ağ Yapısı

Eğitmen: Hüseyin Babal
Süre: 2 Saat (Teori + Hands-on Lab)

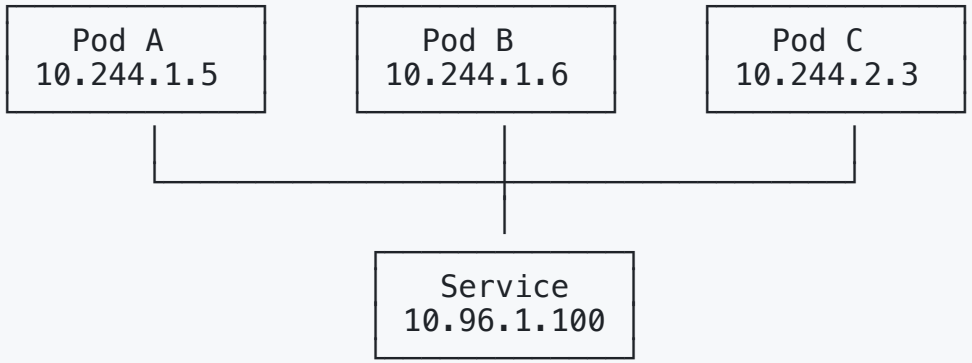
📋 Bu Oturum İçeriği

- ✅ Kubernetes networking fundamentals
- ✅ Service types ve kullanım alanları
- ✅ Ingress controllers ve routing
- ✅ Network policies ve güvenlik
- ✅ Pod-to-pod communication
- ✅ **E-commerce services arası iletişim (Lab)**
- ✅ DNS ve service discovery

🌐 Kubernetes Networking Model

Temel Prensipler:

1. **Her Pod'un unique IP'si** vardır
2. **Pod'lar birbirleriyle** doğrudan iletişim kurabilir
3. **Node'lar Pod'larla** NAT olmadan iletişim kurar
4. **Pod içindeki container'lar** localhost üzerinden iletişim kurar



❌ Pod IP'lerinin Problemleri

Sorunlar:

- **Pod IP'leri değişken** (restart'ta yeni IP)
- **Pod discovery** nasıl yapılacak?
- **Load balancing** manuel
- **External erişim** mümkün değil

Çözüm: Service Abstraction

```
Pod'lar (Ephemeral) + Service (Stable) = ✅
```

Service, Pod'lara **stable network identity** sağlar

🎯 Service Nedir?

> **Service**, Pod'ların önünde duran ve onlara stable network endpoint sağlayan bir abstraction'dır.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: user-service
spec:
  selector:
    app: user-service
  ports:
  - protocol: TCP
    port: 80        # Service port
    targetPort: 8080 # Pod port
  type: ClusterIP
```

Service → Label Selector → Pod'lar

🔍 Service Types

1. ClusterIP (Default)

- **Internal cluster** erişimi
- **Port:** Cluster içi IP
- **Kullanım:** Microservices arası iletişim

2. NodePort

- **External** erişim
- **Port:** 30000-32767 arası
- **Kullanım:** Development/testing

3. LoadBalancer

- **Cloud provider** load balancer
- **Kullanım:** Production external erişim

4. ExternalName

- **DNS CNAME** mapping
- **Kullanım:** External service'leri cluster'a dahil etme

🏠 ClusterIP Service

```yaml
apiVersion: v1
kind: Service
metadata:
  name: user-service-clusterip
spec:
  type: ClusterIP  # Default
  selector:
    app: user-service
  ports:
  - port: 80
    targetPort: 8080
    protocol: TCP
```

Özellikler:

- **Sadece cluster içi** erişim
- **Default service type**
- **Service discovery** için kullanılır
- **DNS name:** `service-name.namespace.svc.cluster.local`

🚪 NodePort Service

```yaml
apiVersion: v1
kind: Service
metadata:
  name: user-service-nodeport
spec:
  type: NodePort
  selector:
    app: user-service
  ports:
  - port: 80
    targetPort: 8080
    nodePort: 30080  # 30000-32767 arası
```

Erişim:

```bash
# Cluster dışından erişim
curl http://<node-ip>:30080
```

Production'da önerilen değil!

☁️ LoadBalancer Service

```yaml
apiVersion: v1
kind: Service
metadata:
  name: user-service-lb
spec:
  type: LoadBalancer
  selector:
    app: user-service
  ports:
  - port: 80
    targetPort: 8080
```

Cloud Provider Integration:

- **AWS:** Application Load Balancer
- **GCP:** Google Cloud Load Balancer
- **Azure:** Azure Load Balancer

🌐 Ingress - Advanced Routing

> **Ingress**, HTTP/HTTPS routing'i yönetir ve cluster'a gelen external traffic'i yönlendirir.

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ecommerce-ingress
spec:
  rules:
  - host: api.ecommerce.com
    http:
      paths:
      - path: /users
        pathType: Prefix
        backend:
          service:
            name: user-service
            port:
              number: 80
      - path: /products
        pathType: Prefix
        backend:
          service:
            name: product-service
            port:
              number: 80
```

🔒 Ingress Features

SSL/TLS Termination

```yaml
spec:
  tls:
  - hosts:
    - api.ecommerce.com
    secretName: ecommerce-tls
```

Path-based Routing

```
api.ecommerce.com/users     → user-service
api.ecommerce.com/products  → product-service
api.ecommerce.com/orders    → order-service
```

Host-based Routing

```
users.ecommerce.com    → user-service
products.ecommerce.com → product-service
orders.ecommerce.com   → order-service
```

🛡️ Network Policies

> **Network Policy**, Pod'lar arası network traffic'i kontrol eden güvenlik kurallarıdır.

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: user-service-netpol
spec:
  podSelector:
    matchLabels:
      app: user-service
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
    - protocol: TCP
      port: 8080
```

🔐 Network Policy Senaryoları

Database Isolation

```yaml
# Sadece backend service'ler DB'ye erişebilsin
ingress:
- from:
  - podSelector:
      matchLabels:
        tier: backend
```

Namespace Isolation

```yaml
# Sadece aynı namespace'ten erişim
ingress:
- from:
  - namespaceSelector:
      matchLabels:
        name: production
```

Default Deny All

```yaml
# Tüm traffic'i reddet (whitelist approach)
spec:
  podSelector: {}  # Tüm pod'lar
  policyTypes:
  - Ingress
```

🧪 Lab: E-commerce Networking

User Service - ClusterIP

```yaml
apiVersion: v1
kind: Service
metadata:
  name: user-service
spec:
  selector:
    app: user-service
  ports:
  - port: 8080
    targetPort: 80
  type: ClusterIP
```

Product Service - ClusterIP

```yaml
apiVersion: v1
kind: Service
metadata:
  name: product-service
spec:
  selector:
    app: product-service
  ports:
  - port: 8080
    targetPort: 80
```

🚀 Lab: Ingress Setup

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ecommerce-api-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: ecommerce.local
    http:
      paths:
      - path: /api/v1/users
        pathType: Prefix
        backend:
          service:
            name: user-service
            port:
              number: 8080
      - path: /api/v1/products
        pathType: Prefix
        backend:
          service:
            name: product-service
            port:
              number: 8080
```

🔍 DNS ve Service Discovery

Service DNS Names

```
# Tam DNS ismi
user-service.default.svc.cluster.local

# Aynı namespace'te kısa isim
user-service

# Cross-namespace erişim
user-service.production.svc.cluster.local
```

DNS Resolution Test

```
kubectl run debug --image=busybox --rm -it -- sh
nslookup user-service
nslookup user-service.default.svc.cluster.local
```

📡 Service Discovery Lab

Pod İçinden Service Test

```
# Debug pod oluştur
kubectl run curl-test --image=curlimages/curl --rm -it -- sh

# Service'leri test et
curl http://user-service:8080/api/v1/health
curl http://product-service:8080/api/v1/health
curl http://order-service:8080/api/v1/health
```

Environment Variables

```
# Pod içinde service environment variables
env | grep SERVICE
```

🌐 Lab: Inter-Service Communication

```yaml
# Test pod for service communication
apiVersion: v1
kind: Pod
metadata:
  name: service-test
spec:
  containers:
  - name: curl
    image: curlimages/curl
    command: ["/bin/sh"]
    args: ["-c", "while true; do sleep 30; done"]
    env:
    - name: USER_SERVICE_URL
      value: "http://user-service:8080"
    - name: PRODUCT_SERVICE_URL
      value: "http://product-service:8080"
```

Communication Test

```bash
kubectl exec -it service-test -- curl $USER_SERVICE_URL/health
kubectl exec -it service-test -- curl $PRODUCT_SERVICE_URL/health
```

📝 Sonraki Oturum Önizlemesi

Session 05: Durum Yönetimi ve Veri Kalıcılığı

- **StatefulSets** vs Deployments
- **Persistent Volumes** (PV) ve **Persistent Volume Claims** (PVC)
- **Storage Classes** ve dynamic provisioning
- **Database deployment** best practices
- **E-commerce veritabanları** için persistent storage

Hazırlık:

- Service networking kavramlarını pekiştirin
- Database fundamentals gözden geçirin

🙏 Teşekkürler!

Network Master! 🌐

Bugün öğrendiklerimiz:
✅ Service types'ları öğrendik
✅ Ingress ile routing yaptık
✅ Network policies ile güvenlik sağladık
✅ Microservices arası iletişim kurduk

İletişim:

- GitHub: @huseyinbabal

Sonraki Oturum: Session 05 - Durum Yönetimi ve Veri Kalıcılığı